

Don Bosco Institute of Technology, Kurla
Academic Year 2025-26
EXPERIMENT NO: 6

Name of the student: Shraddha Gajanan Desai

Roll No: 18

Division: A

Batch: A

Date of Performance: 10/03/2026

Date of Submission:15/03/2026

Aim of the Experiment:	To simulate and understand the working of a SQL Injection attack on a vulnerable web application.
Learning Objectives:	• To understand the concept of SQL Injection attacks. • To analyze how improper input validation can compromise database security. • To observe how attackers bypass authentication using malicious SQL queries.
Lab Outcome s:	Student will able to • ☐ Understand common web application vulnerabilities. • ☐ Demonstrate how SQL Injection can be used to gain unauthorized access to a database. • ☐ Apply preventive techniques to secure web applications against SQL Injection attacks.
Tools & Software Required :	• XAMPP / WAMP Server • MySQL Database • Web Browser • PHP

Don Bosco Institute of Technology, Kurla
Academic Year 2025-26

Theory:	SQL Injection (SQLi) SQL Injection is a type of vulnerability that occurs when an attacker can interfere with the queries that an application makes to its database. It typically allows an attacker to view data they are not normally able to retrieve. This can include data belonging to other users, or any other data that the application itself can access. Authentication Bypass In an Authentication Bypass, attackers use malicious SQL queries to trick the application into logging them in without a valid password. • For example, using a tautology like ' OR '1'='1 creates a logical condition that is always true.
----------------	--

	• When the database evaluates this, the WHERE clause becomes true for the first user in the table (usually the admin), granting the attacker access. Input Validation Input Validation is the process of ensuring that a program operates on clean, correct, and useful data. • The experiment demonstrates how improper input validation can compromise database security. • When user-supplied data is directly concatenated into a SQL command without being sanitized or "escaped," the data can be misinterpreted by the database as part of the command itself.
--	--

Don Bosco Institute of Technology, Kurla
Academic Year 2025-26

Algorithm:	Step 1: Install and start XAMPP/WAMP server and run Apache and MySQL services. Step 2: Create a database using MySQL. <pre>CREATE DATABASE securitylab;</pre> USE securitylab; Step 3: Create a table for storing user credentials. <pre>CREATE TABLE users(id INT(4)PRIMARY KEY ,username VARCHAR(50), password VARCHAR(50));</pre> Step 4: Insert sample data. <pre>INSERT INTO users(username,password) VALUES('admin','admin123');</pre> Step 5: Create a login form (login.html). <pre><html> <head><title>SQL ENJECTION</title></head> <body> <form action="login.php" method="post"> Username: <input type="text" name="username">

 Password: <input type="text" name="password">

 <input type="submit" value="Login"> </form> </body></pre>
-------------------	---

Don Bosco Institute of Technology, Kurla
Academic Year 2025-26

</html>

Step 6: Create a vulnerable PHP login script (login.php).

```
<?php
$conn = mysqli_connect("localhost","root",""," securitylab");
if(!$conn){ die("Connection failed: " . mysqli_connect_error()); }
$username = $_POST['username'];
$password = $_POST['password'];
$sql = "SELECT * FROM users WHERE username='$username' AND password='$password'";
$result = mysqli_query($conn, $sql);
if(!$result){ die("Query Error: " . mysqli_error($conn)); }
if(mysqli_num_rows($result) > 0){ echo "<h2 style='color:green;'>Login Successful</h2>"; $allData = mysqli_query($conn, "SELECT * FROM users"); echo "<h3>All Users Table Data:</h3>"; echo "<table border='1' cellpadding='10'>"; echo "<tr><th>ID</th><th>Username</th><th>Password</th></tr>"; while($row = mysqli_fetch_assoc($allData)){ echo "<tr>"; echo "<td>".$row['id']. "</td>"; echo "<td>".$row['username']. "</td>"; echo "<td>".$row['password']. "</td>"; echo "</tr>"; } echo "</table>"; } else { echo "<h2 style='color:red;'>Invalid Credentials</h2>"; }
echo "<br><br><b>Generated Query:</b> " . $sql;
mysqli_close($conn);
?>
```

Step 7: Run the login page in the browser.

<http://localhost/login.html>

Step 8:

Enter valid credentials: admin / admin123

Step 9: Perform SQL Injection by entering:

1. Username: admin Password: ' OR '1'='1
2. Username: admin' -- Password: anything

Don Bosco Institute of Technology, Kurla
Academic Year 2025-26

	Step 10: Observe that the login is successful even without the correct password.
	Login.html <pre><!DOCTYPE html> <html> <head> <title>SQL Injection</title> </head> <body> <h3>Name: Shraddha Desai ; Roll No: 18</h3> <h2>Login Form</h2> <form action="login.php" method="post"> Username: <input type="text" name="username">

 Password: <input type="text" name="password">

 <input type="submit" value="Login"> </form> </body> </html></pre>

Don Bosco Institute of Technology, Kurla
Academic Year 2025-26

Login.php

```
<?php
$conn = mysqli_connect("localhost","root","", "securitylab");

if(!$conn){
    die("Connection failed: " . mysqli_connect_error());
}

$username = $_POST['username'];
$password = $_POST['password'];

// ☒ SECURE METHOD: Prepared Statement
$stmt = $conn->prepare("SELECT * FROM users WHERE username=? AND
password=?");
$stmt->bind_param("ss", $username, $password);
$stmt->execute();

$result = $stmt->get_result();

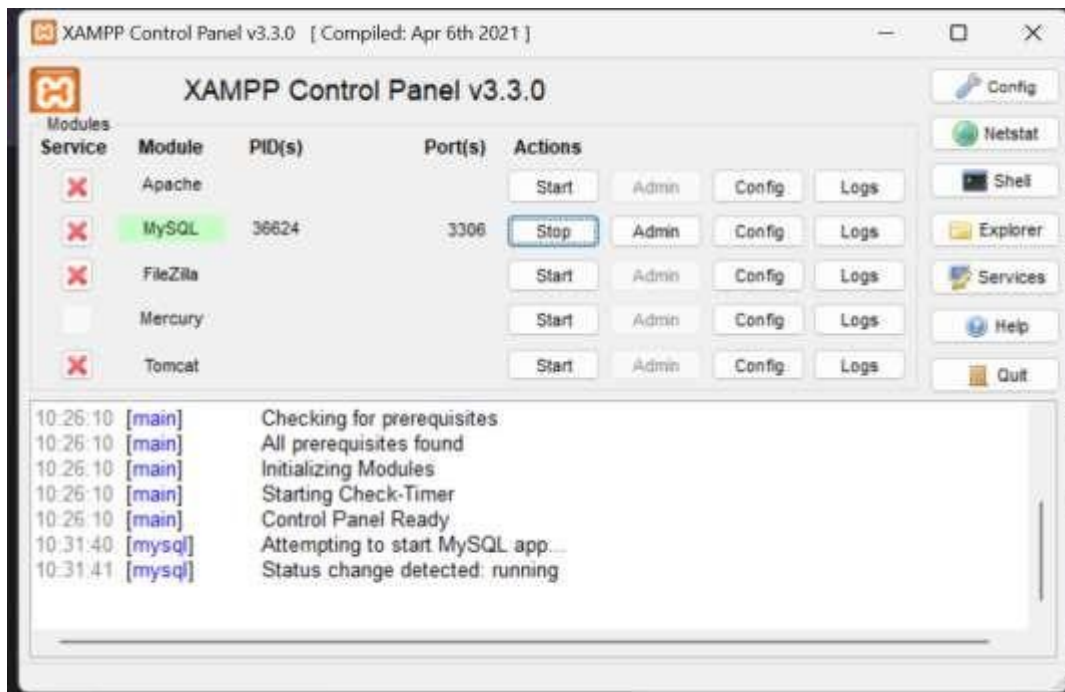
if($result->num_rows > 0){
    echo "<h2 style='color:green;'>Login Successful (Secure Mode)</h2>";
} else {
    echo "<h2 style='color:red;'>Invalid Credentials</h2>";
    echo "<p>SQL Injection attack blocked using prepared statements.</p>";
}

mysqli_close($conn);
?>
```

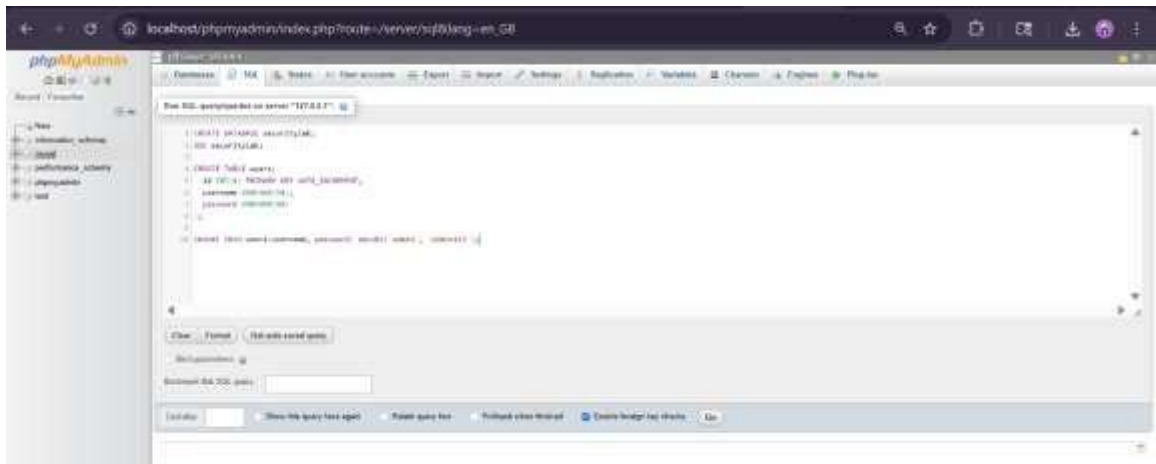
Don Bosco Institute of Technology, Kurla
Academic Year 2025-26

Output:

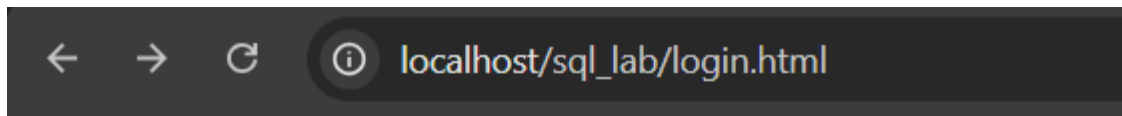
XAMPP Services Running



Database and Table Creation



Login Page Interface



Name: Shraddha Desai ; Roll No: 18

Login Form

Username:

Password:

Valid Admin Login Result

← → ↻ ⓘ localhost/sql_lab/login.php

Name: Shraddha Desai ; Roll No: 18

Login Successful

Query: SELECT * FROM users WHERE username='admin' AND password='admin123'

Name: Shraddha Desai ; Roll No: 18

Login Form

Username:

Password:

Don Bosco Institute of Technology, Kurla
Academic Year 2025-26

Without prepared statements

Name: Shraddha Desai ; Roll No: 18

Login Successful

Query: SELECT * FROM users WHERE username='admin' AND password="" OR '1'='1'

with prepared statements

Invalid Credentials

SQL Injection attack blocked using prepared statements.

Don Bosco Institute of Technology, Kurla
Academic Year 2025-26

Conclusion:	This experiment demonstrated how SQL Injection can bypass authentication mechanisms. By injecting malicious input like ' OR '1'='1, the SQL query condition becomes true, allowing unauthorized access. This highlights the importance of input validation and secure coding practices such as prepared statements.
--------------------	--